פונקציית התמורה - "permutations" (ובקיצור "P")

קלט הפונקציה הינו ה - "State" (בקוד נקרא "STATE_ INPUT"). גודלו 320 סיביות, והוא מורכב מחמישה מילים שגודל כל אחד מהם 64 בית:

$$State = S0 \parallel S1 \parallel S2 \parallel S3 \parallel S4$$

S0 - וקטור אתחול ה IV.

S1 - חצי ראשון של המפתח - K.

S2 - חצי שני של המפתח - K.

S3 - חצי ראשון של ה-Nonce.

S4 - חצי שני של ה-Nonce.

הפונקציה מבצעת מספר סיבובים - "Rounds" (ובקיצור "rnd") הנע בטוח: $1 \leq rnd \leq 16$

כל סיבוב בפונקציה מורכב משלושה שלבים עיקריים, המתבצעים בזה אחר זה:

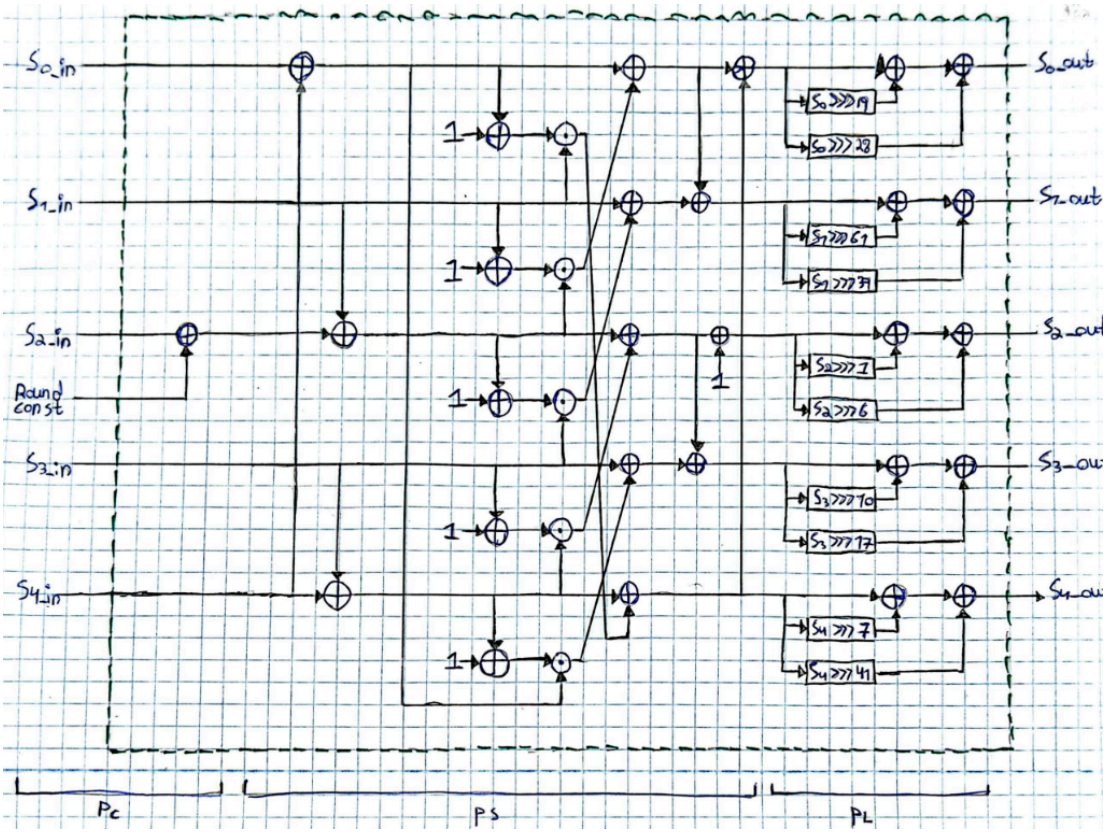
$$p = pL \circ pS \circ pC$$

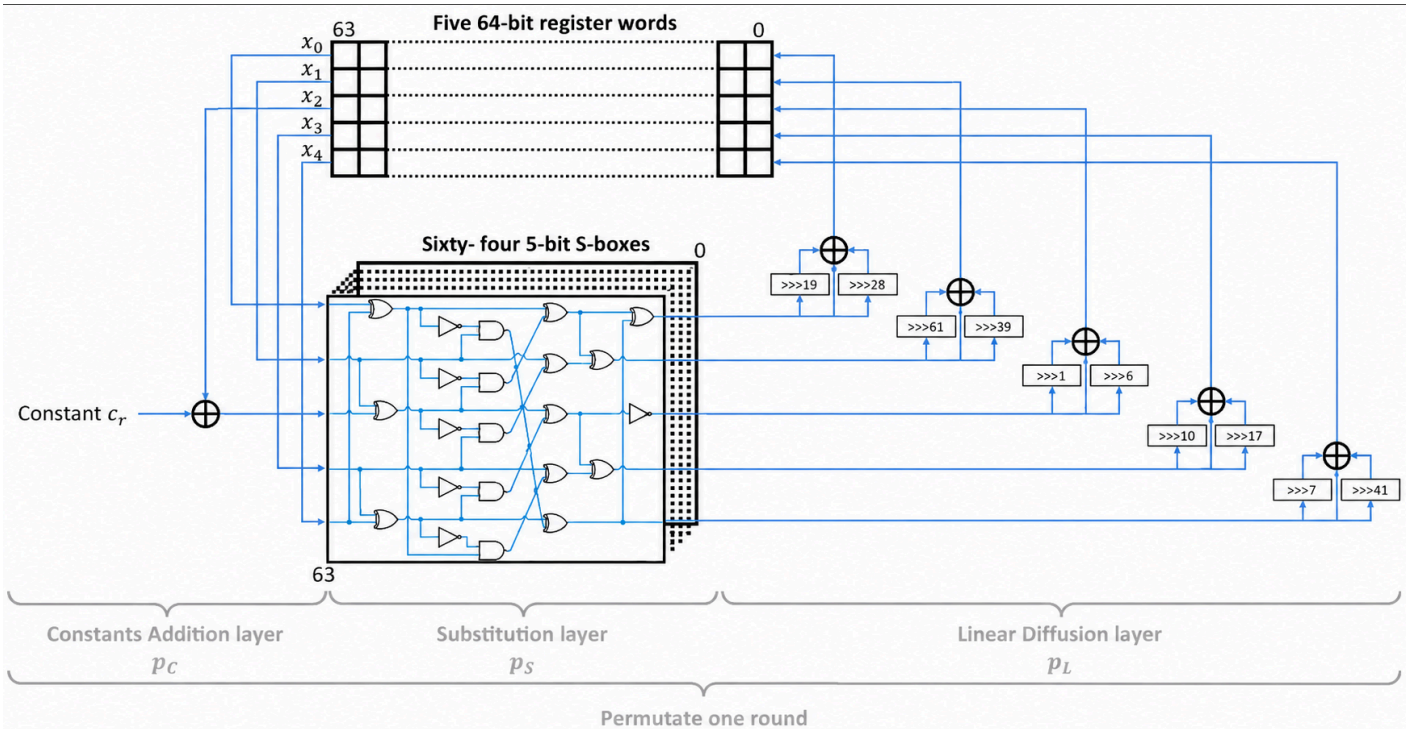
1. **pC** – שכבת הוספת הקבועים - Constant Addition Layer.
2. **pS** – שכבת ההחלפה - Substitution Layer.
3. **pL** – שכבת הדיפוזיה הליניארית - Linear Diffusion Layer.

בס"ד

המרכז האקדמי לב

סכמות המסכמות את שלושת השלבים ביחד:





שלושת השלבים בהרחבה:

1. שלב ראשון - הוספת הקבועים:

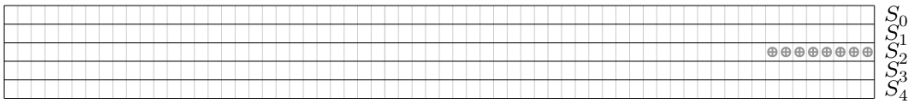
ביצוע XOR בין בין המילה S2 שב - $State$ לבין מספר קבוע, אחד מאלה המופיעים בטבלה הבאה:

i	$\texttt{const}_i$	i	$\texttt{const}_i$
0	0x000000000000003c	8	0x00000000000000b4
1	0x000000000000002d	9	0x00000000000000a5
2	0x000000000000001e	10	0x0000000000000096
3	0x000000000000000f	11	0x0000000000000087
4	0x00000000000000f0	12	0x0000000000000078
5	0x00000000000000e1	13	0x0000000000000069
6	0x00000000000000d2	14	0x000000000000005a
7	0x00000000000000c3	15	0x000000000000004b

המספר הספציפי נקבע ע"פ הנוסחה הבאה:

$$Ci = const_{16-rnd+i}$$

אורך כל קבוע הינו 64 ביט, אך כיוון ש- 56 הביטים הראשונים שלו הם אפס, בפועל פעולת ה־XOR משפיעה רק על 8 הביטים ה LSB˙ (הימניים ביותר) של S2, ו- 56 הביטים הנותרים נשארים ללא שינוי.

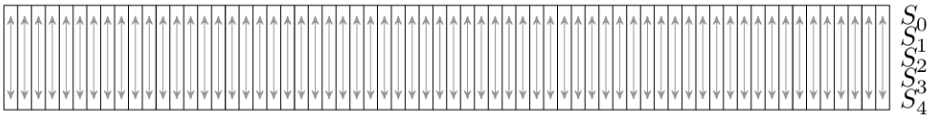


מטרתו של כל שלב זה הינה יצירת הבדלים בין כל סיבוב ליתר הסיבובים.

סיגנל המוצא לאחר פעולה זו נקרא בקיצור **pc** (כלומר **p**ermutation **c**onstan - המצב אחרי הוספת הקבוע).

2. שלב שני - שכבת ההחלפה:

בהתייחס ל- State כמסודר בצורה "אנכית" באופן הבא:

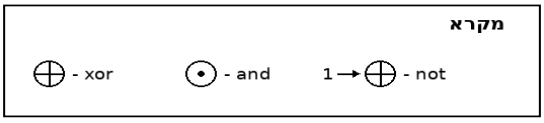


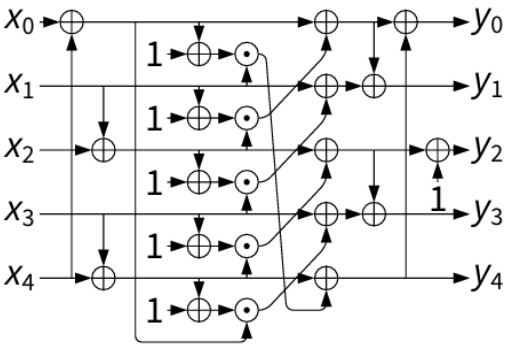
נוצרות למעשה 64 קבוצות, כל קבוצה בעלת 5 ביטים בעלי אותו אינדקס המורכבת מחמשת המילים של ה State (ביט אחד מכל מילה).

בו זמנית עבור כל אחת מ- 64 הקבוצות, באופן מקביל, נכנסים כל חמישית ביטים ל"קופסה שחורה" שנקראת **S-Box** בה מחלפים את הביטים שנכנסו Xi, ומייצרים במוצא חמישה ביטים אחרים Yi.

$$(y0,\ldots,y4) = SBOX(x0,\ldots,x4)$$

השינוי נקבע ע"פ הסכמה הבאה:





כאשר X0,X1,X2,X3,X4 מוגדרים באופן הבא:

```
x0 := S0_pc xor S4_pc;
x1 := S1_pc;
x2 := S2_pc xor S1_pc;
x3 := S3_pc;
x4 := S4_pc xor S3_pc;
```

או בעזרת הטבלה הבאה בה מפורשים כל 32 הצירופים האפשריים:

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
SBox(x)	4	b	1f	14	1a	15	9	2	1b	5	8	12	1d	3	6	1c
x	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e	1f
SBox(x)	1e	13	7	e	0	d	11	18	10	c	1	19	16	a	f	17

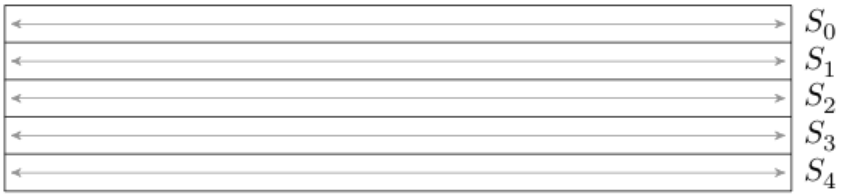
מטרת שלב זה הינה ליצור "ערבוב" כך ששינוי של ביט אחד בקלט ישפיע על המון ביטים בפלט. פעולה זו יוצרת ערבוב לא-ליניארי חזק (מתמטיקה מורכבת)

לסיגנל המוצא לאחר פעולה זו נקרא בקיצור **ps** (כלומר **p**ermutation **s**ubstitution - המצב אחרי ה- SBox).

המרכז האקדמי לב

3. שלב שלישי - שכבת הפיזור ("דיפוזיה") הליניארית:

בשלב זה אנו מבצעים פיזור וערבוב בתוך כל אחת מ-5 המילים (S0….S4) בנפרד.



המטרה היא לגרום לכך ששינוי של ביט אחד יחיד יתפשט במהירות וישפיע על כל שאר הביטים בתוך המילה.

לכל אחת מ-5 המילים יש נוסחת הזזה ייחודית משלה:

$$\Sigma_0(S_0) = S_0 \oplus (S_0 \ggg 19) \oplus (S_0 \ggg 28)$$
$$\Sigma_1(S_1) = S_1 \oplus (S_1 \ggg 61) \oplus (S_1 \ggg 39)$$
$$\Sigma_2(S_2) = S_2 \oplus (S_2 \ggg 1) \oplus (S_2 \ggg 6)$$
$$\Sigma_3(S_3) = S_3 \oplus (S_3 \ggg 10) \oplus (S_3 \ggg 17)$$
$$\Sigma_4(S_4) = S_4 \oplus (S_4 \ggg 7) \oplus (S_4 \ggg 41)$$

בעזרת ביצוע XOR של כל מילה עם שתי גרסאות "מוזזות"[1] של עצמה, אנו מזיזים את הביטים ומפזרים אותם לכל אורך ה-State.

למוצא לאחר פעולה זו נקרא: " STATE_OUTPUT ".

[1] לכל מילה יש מרחקי הזזה שונים, שנקבעו כדי למקסם את הערבוב. (סימון הזזה: <<<).

בס"ד



קידוד פונקצית התמורה:

```vhdl
library IEEE;
    use IEEE.std_logic_1164.all;

entity ASCON_ROUND is
    port
    (
        ROUND_CONST : in std_logic_vector(7 downto 0);                              -- 8-bit round constant input to differentiate rounds
        S0_in, S1_in, S2_in, S3_in, S4_in : in std_logic_vector(63 downto 0);       -- 64-bit input state words (Total 320 bits)
        S0_out, S1_out, S2_out, S3_out, S4_out : out std_logic_vector(63 downto 0)  -- 64-bit output state words after 1 round
    );
end ASCON_ROUND;

architecture Behavioral of ASCON_ROUND is
    signal S0_pc, S1_pc, S2_pc, S3_pc, S4_pc : std_logic_vector(63 downto 0);       -- State after constant-addition layer pC.
    signal S0_ps, S1_ps, S2_ps, S3_ps, S4_ps : std_logic_vector(63 downto 0);       -- State after substitution layer pS.

begin
    ------------------------------------------------------------------------
    -- 1. CONSTANT ADDITION pC
    ------------------------------------------------------------------------
    S0_pc <= S0_in;                                                             -- pC does not change S0.
    S1_pc <= S1_in;                                                             -- pC does not change S1.
    S2_pc <= S2_in(63 downto 8) & (S2_in(7 downto 0) xor ROUND_CONST);          --   S2[7:0] <- S2[7:0] XOR round_constant The remaining bits are unchanged.
    S3_pc <= S3_in;                                                             -- pC does not change S3.
    S4_pc <= S4_in;                                                             -- pC does not change S4.

    ------------------------------------------------------------------------
    -- 2. SUBSTITUTION LAYER pS
    ------------------------------------------------------------------------
    process(S0_pc, S1_pc, S2_pc, S3_pc, S4_pc)                                  -- Combinational bit-sliced S-box, applied to 64 slices in parallel.
        variable x0, x1, x2, x3, x4 : std_logic_vector(63 downto 0);            -- Working vectors for the five S-box bit planes.
        variable t0, t1, t2, t3, t4 : std_logic_vector(63 downto 0);            -- Temporary vectors for the non-linear S-box terms.
    begin

        x0 := S0_pc xor S4_pc;                                                  -- S-box input affine step.
        x1 := S1_pc;                                                            -- S1 enters this step unchanged.
        x2 := S2_pc xor S1_pc;                                                  -- S-box input affine step.
        x3 := S3_pc;                                                            -- S3 enters this step unchanged.
        x4 := S4_pc xor S3_pc;                                                  -- S-box input affine step.

        t0 := x0 xor ((not x1) and x2);                                         -- Non-linear chi term for output plane 0.
        t1 := x1 xor ((not x2) and x3);                                         -- Non-linear chi term for output plane 1.
        t2 := x2 xor ((not x3) and x4);                                         -- Non-linear chi term for output plane 2.
        t3 := x3 xor ((not x4) and x0);                                         -- Non-linear chi term for output plane 3.
        t4 := x4 xor ((not x0) and x1);                                         -- Non-linear chi term for output plane 4.

        t1 := t1 xor t0;                                                        -- Final S-box affine mixing.
        t0 := t0 xor t4;                                                        -- Final S-box affine mixing.
        t3 := t3 xor t2;                                                        -- Final S-box affine mixing.
        t2 := not t2;                                                           -- Final S-box inversion required by Ascon.

        S0_ps <= t0;                                                            -- Store S-box output S0.
        S1_ps <= t1;                                                            -- Store S-box output S1.
        S2_ps <= t2;                                                            -- Store S-box output S2.
        S3_ps <= t3;                                                            -- Store S-box output S3.
        S4_ps <= t4;                                                            -- Store S-box output S4.
    end process;

    ------------------------------------------------------------------------
    -- 3. LINEAR DIFFUSION LAYER pL
    ------------------------------------------------------------------------
    S0_out <= S0_ps xor (S0_ps(18 downto 0) & S0_ps(63 downto 19)) xor (S0_ps(27 downto 0) & S0_ps(63 downto 28));     -- S0 = S0 XOR ROTR(S0,19) XOR ROTR(S0,28)
    S1_out <= S1_ps xor (S1_ps(60 downto 0) & S1_ps(63 downto 61)) xor (S1_ps(38 downto 0) & S1_ps(63 downto 39));     -- S1 = S1 XOR ROTR(S1,61) XOR ROTR(S1,39)
    S2_out <= S2_ps xor (S2_ps(0) & S2_ps(63 downto 1)) xor (S2_ps(5 downto 0) & S2_ps(63 downto 6));                  -- S2 = S2 XOR ROTR(S2, 1) XOR ROTR(S2, 6)
    S3_out <= S3_ps xor (S3_ps(9 downto 0) & S3_ps(63 downto 10)) xor (S3_ps(16 downto 0) & S3_ps(63 downto 17));     -- S3 = S3 XOR ROTR(S3,10) XOR ROTR(S3,17)
    S4_out <= S4_ps xor (S4_ps(6 downto 0) & S4_ps(63 downto 7)) xor (S4_ps(40 downto 0) & S4_ps(63 downto 41));      -- S4 = S4 XOR ROTR(S4, 7) XOR ROTR(S4,41)
end architecture Behavioral;
```